

Lippens_Charles_2_clustering_visualisations_072026

August 14, 2026

1 Projet 11 - Étude de marché Python

1.1 Notebook 2 : ACP, clustering et visualisations

Auteur : Charles Lippens **Date** : juillet 2026 **Prérequis** : notebook 1 exécuté (Lippens_Charles_1_preparation_nettoyage_analyse_exploratoire_072026.ipynb), qui produit dataset_p11_clean.csv (172 pays, 13 variables analytiques depuis le retrait de la disponibilité de protéines, décidé en session après lecture de la matrice de corrélations) et fx_par_pays.csv (taux de change par pays).

1.1.1 Objectif

Identifier des groupes de pays cibles pour l'export de poulet bio et en tirer une courte liste de marchés prioritaires, à partir du dataset préparé au notebook 1.

1.1.2 Contraintes

Données ouvertes uniquement (FAO 2017, Banque mondiale) ; au moins 100 pays et 60 % de la population mondiale couverts (ici 172 pays, 97,6 %) ; au moins 8 variables analytiques (ici 13, plus le taux de change) ; une ACP puis deux méthodes de clustering (CAH et K-Means) comparées ; zéro valeur manquante en entrée.

1.1.3 Sommaire

1. Imports et configuration - 2. Chargement du dataset et du taux de change - 3. Variables actives et standardisation - 4. Réalisation de l'ACP (4.1 choix du nombre de composantes) - 5. Cercle des corrélations - 6. Projection des pays - 7. Contributions des variables - 8. Sauvegarde des coordonnées factorielles - 9. CAH - 10. K-Means - 11. Clusters sur le plan factoriel - 12. Comparaison CAH / K-Means (12.1 contrôle du change) - 13. Caractérisation des clusters - 14. Recommandation et top 5 - 15. Carte choroplèthe - 16. Export du dataset étiqueté - 17. Contrôle des livrables.

1.1.4 Cheminement

1. Charger le dataset final et joindre le taux de change par pays : le change varie pays par pays, il entre donc dans la préparation du clustering.
2. Standardiser, réaliser l'ACP : éboullis, cercle des corrélations, projection, contributions.
3. CAH puis K-Means, comparaison des deux méthodes, contrôle de l'effet du change sur la partition.
4. Caractériser les clusters, désigner la cible (scoring recalibré), carte et export.

1.2 1. Imports et configuration

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
import warnings
warnings.filterwarnings('ignore')

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score,
↳normalized_mutual_info_score
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster

sns.set_theme(style='whitegrid', context='notebook')
plt.rcParams['figure.figsize'] = (10, 7)
plt.rcParams['axes.titlesize'] = 13

# racine du projet : on part du dossier courant et on remonte tant qu'on ne
↳trouve pas
# le dossier data/processed, celui que le notebook 1 alimente. Le notebook
↳s'exécute donc
# quel que soit l'endroit où il est range dans l'arborescence livrée.
RACINE = next((d for d in [Path.cwd(), *Path.cwd().parents] if (d / 'data' /
↳'processed').is_dir()), None)
if RACINE is None:
    raise FileNotFoundError(
        "dossier data/processed introuvable en remontant depuis " + Path.cwd().
↳name
        + " : exécuter d'abord le notebook 1, ou déposer ce notebook dans le
↳dossier décompressé"
    )

DATA_PROCESSED = RACINE / 'data' / 'processed'
FIG = RACINE / 'outputs' / 'figures'
OUT = RACINE / 'outputs'
FIG.mkdir(parents=True, exist_ok=True)

print(f'Environnement prêt. Racine du projet détectée : {RACINE.name}')
```

Environnement prêt. Racine du projet détectée :
Etude_de_marche_Python_Lippens_Charles

1.3 2. Chargement : dataset final et taux de change, dans le cheminement

Le taux de change varie pays par pays : il rejoint donc les variables du clustering dès la préparation, comme convenu en session. Passage au logarithme, car le niveau brut d'un taux est un artefact de dénomination (185 yens et 0,85 livre ne disent rien de la santé des monnaies). Zone euro : taux 1, logarithme 0, ce qui se lit comme une absence de risque de change pour un exportateur français.

```
[2]: df = pd.read_csv(DATA_PROCESSED / 'dataset_p11_clean.csv')

# la disponibilité de protéines était corrélée à 0,99 avec la version en kilos :
#   ↳ c'était la même
# information à une conversion d'unité près. On l'avait retirée au notebook 1
#   ↳ après lecture de la
# matrice de corrélations. Si le CSV n'a pas encore été régénéré, je refais le
#   ↳ retrait ici pour
# rester sur mes 13 variables structurelles.
cols_prot = [c for c in df.columns if 'disponibilit' in c.lower() and 'prot' in
#   ↳ c.lower()]
if cols_prot:
    df = df.drop(columns=cols_prot)
    print('Retrait de la disponibilité de protéines (redondante avec la version
#   ↳ en kilos)')

print(f'Dataset : {len(df)} pays, {df.shape[1]} colonnes ({df.shape[1] - 2}
#   ↳ variables analytiques)')

# taux de change par pays, produit par le notebook 1 (section 8)
fx = pd.read_csv(DATA_PROCESSED / 'fx_par_pays.csv')
df = df.merge(fx[['iso3', 'devise', 'taux_pour_1_eur']], on='iso3', how='left')

# log du taux : atténue l'artefact de dénomination (185 yens et 0,85 livre ne
#   ↳ se comparent pas bruts)
df['fe_change_log'] = np.log(df['taux_pour_1_eur'])

# les rares devises non cotées reçoivent la médiane (ici la table couvre tous
#   ↳ les pays)
manquants = df['fe_change_log'].isna().sum()
df['fe_change_log'] = df['fe_change_log'].fillna(df['fe_change_log'].median())
print(f'Taux joint sur iso3 : {df["taux_pour_1_eur"].notna().sum()} pays cotés,
#   ↳ {manquants} imputés à la médiane')
print(f'Zone euro (log = 0) : {(df["fe_change_log"] == 0).sum()} pays')
df[['pays', 'devise', 'taux_pour_1_eur', 'fe_change_log']].head()
```

Dataset : 172 pays, 15 colonnes (13 variables analytiques)

Taux joint sur iso3 : 171 pays cotés, 1 imputés à la médiane

Zone euro (log = 0) : 21 pays

```
[2]:
```

	pays	devise	taux_pour_1_eur	fe_change_log
0	Afghanistan	AFN	74.942984	4.316728
1	South Africa	ZAR	19.096906	2.949526
2	Albania	ALL	93.676976	4.539852
3	Algeria	DZD	151.924900	5.023386
4	Germany	EUR	1.000000	0.000000

1.4 3. Sélection des variables actives et standardisation

13 variables structurelles plus le change en log : 14 variables actives. On écarte les identifiants, la devise (texte) et le taux brut (remplacé par son log).

Normalisation retenue : log d'abord pour les variables multiplicatives (population, potentiel de marché, change), puis centrage-réduction (z-score) pour que chaque variable compte à égalité, quelle que soit son unité. C'est le prérequis de l'ACP, qui travaille sur les corrélations. Concrètement, on centre et on réduit chaque variable avec le StandardScaler : toutes sont ramenées dans des échelles de grandeur similaires, moyenne 0 et écart-type 1.

```
[3]: # variables actives = tout sauf identifiants, devise texte et taux brut (on
      ↪garde son log)
vars_actives = [c for c in df.columns if c not in ['iso3', 'pays', 'devise',
      ↪'taux_pour_1_eur']]
# attendu : 14 variables (13 structurelles + fe_change_log)
print(f'{len(vars_actives)} variables actives :')
for v in vars_actives: print(f' - {v}')

X = df[vars_actives].copy()

# Transformation log pour les variables très asymétriques (population)
for c in ['population_2017_milliers', 'fe_potentiel_marche_musd']:
    if c in X.columns:
        X[c] = np.log1p(X[c])

# Standardisation
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=vars_actives, index=df['pays'])

print(f'\nVérification standardisation : moyenne {X_scaled.mean().mean():.4f},
      ↪std {X_scaled.std().mean():.4f}')
```

14 variables actives :

- viande_de_volailles__disponibilité_alimentaire_en_quantité
- viande_de_volailles__importations_quantité
- viande_de_volailles__production
- population_2017_milliers
- pib_par_hab_usd
- stabilite_politique
- croissance_pib

- inflation
- fe_taux_dependance_import
- fe_intensite_proteique
- fe_potentiel_marche_musd
- fe_import_par_hab
- fe_croissance_pop_10ans
- fe_change_log

Vérification standardisation : moyenne 0.0000, std 1.0029

1.5 4. Réalisation de l'ACP

```
[4]: pca = PCA(n_components=len(vars_actives))
X_pca = pca.fit_transform(X_scaled)

inertie = pca.explained_variance_ratio_ * 100
inertie_cum = inertie.cumsum()

tab_inertie = pd.DataFrame({
    'CP': [f'CP{i+1}' for i in range(len(inertie))],
    'Valeur propre': pca.explained_variance_.round(3),
    'Inertie (%)': inertie.round(2),
    'Inertie cumulée (%)': inertie_cum.round(2),
})
tab_inertie
```

```
[4]:
```

	CP	Valeur propre	Inertie (%)	Inertie cumulée (%)
0	CP1	3.713	26.37	26.37
1	CP2	2.758	19.59	45.96
2	CP3	1.493	10.60	56.56
3	CP4	1.249	8.87	65.43
4	CP5	0.945	6.71	72.14
5	CP6	0.909	6.45	78.59
6	CP7	0.646	4.59	83.18
7	CP8	0.590	4.19	87.37
8	CP9	0.563	4.00	91.37
9	CP10	0.492	3.49	94.86
10	CP11	0.331	2.35	97.21
11	CP12	0.185	1.31	98.52
12	CP13	0.160	1.14	99.66
13	CP14	0.048	0.34	100.00

1.5.1 4.1 Choix du nombre de composantes (éboulis des valeurs propres)

```
[5]: fig, ax = plt.subplots(figsize=(11, 6))
x_axis = np.arange(1, len(inertie) + 1)
ax.bar(x_axis, inertie, color='#2E75B6', alpha=0.7, label='Inertie_
↳individuelle')
```

```

ax.set_xlabel('Composante principale')
ax.set_ylabel('Inertie expliquée (%)', color='#2E75B6')
ax.tick_params(axis='y', labelcolor='#2E75B6')
ax.set_xticks(x_axis)

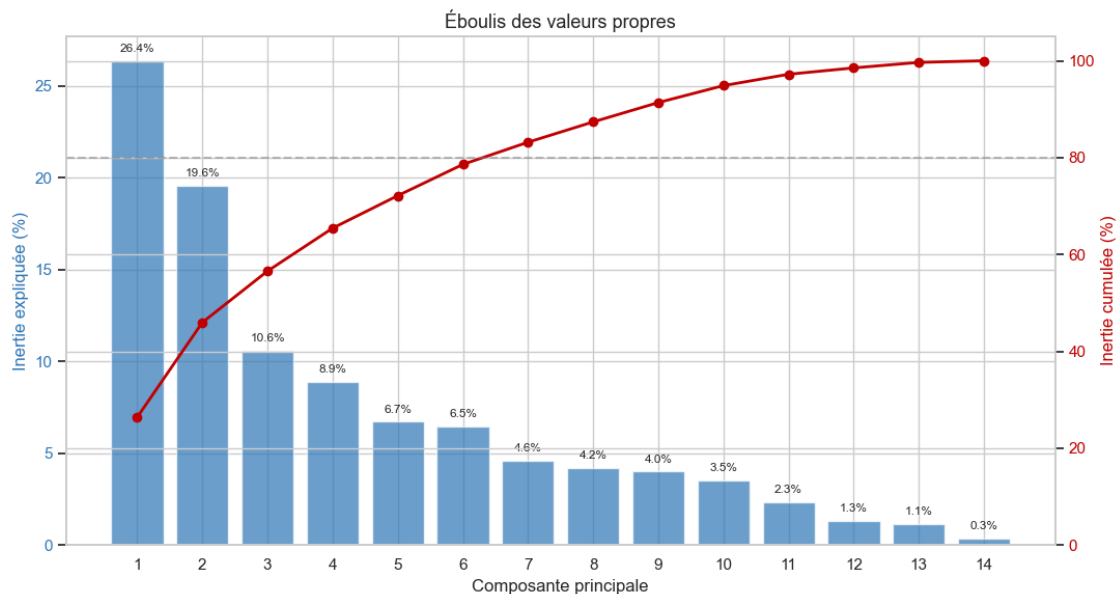
ax2 = ax.twinx()
ax2.plot(x_axis, inertie_cum, marker='o', color='#C00000', linewidth=2,
        label='Inertie cumulée')
ax2.axhline(80, color='gray', linestyle='--', alpha=0.5, label='Seuil 80 %')
ax2.set_ylabel('Inertie cumulée (%)', color='#C00000')
ax2.tick_params(axis='y', labelcolor='#C00000')
ax2.set_ylim(0, 105)

for i, (ind, cum) in enumerate(zip(inertie, inertie_cum)):
    ax.text(i+1, ind + 0.5, f'{ind:.1f}%', ha='center', fontsize=8.5)

plt.title('Éboulis des valeurs propres')
plt.tight_layout()
plt.savefig(FIG / '06_eboulis.png', dpi=120, bbox_inches='tight')
plt.show()

# Détermination du nombre de CP à conserver
n_cp_kaiser = sum(pca.explained_variance_ > 1)
n_cp_80 = int(np.argmax(inertie_cum >= 80) + 1)
print(f'\nCritère de Kaiser (val. propre > 1) : {n_cp_kaiser} CP')
print(f'\nCritère 80 % d'inertie : {n_cp_80} CP")
n_cp = max(min(n_cp_kaiser, 5), 3)
print(f'\nNombre de CP retenues pour le clustering : {n_cp}')

```



Critère de Kaiser (val. propre > 1) : 4 CP
Critère 80 % d'inertie : 7 CP

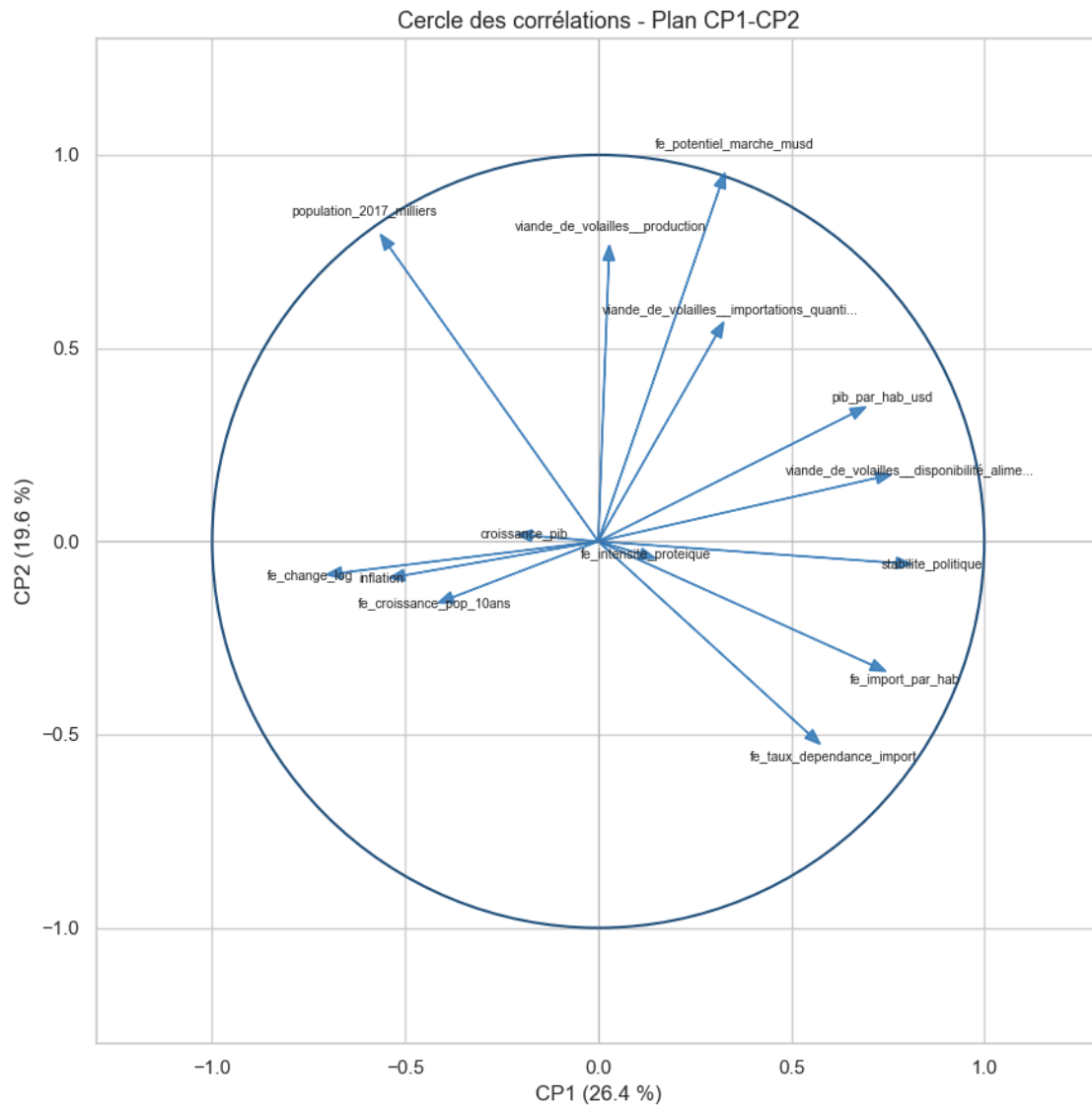
Nombre de CP retenues pour le clustering : 4

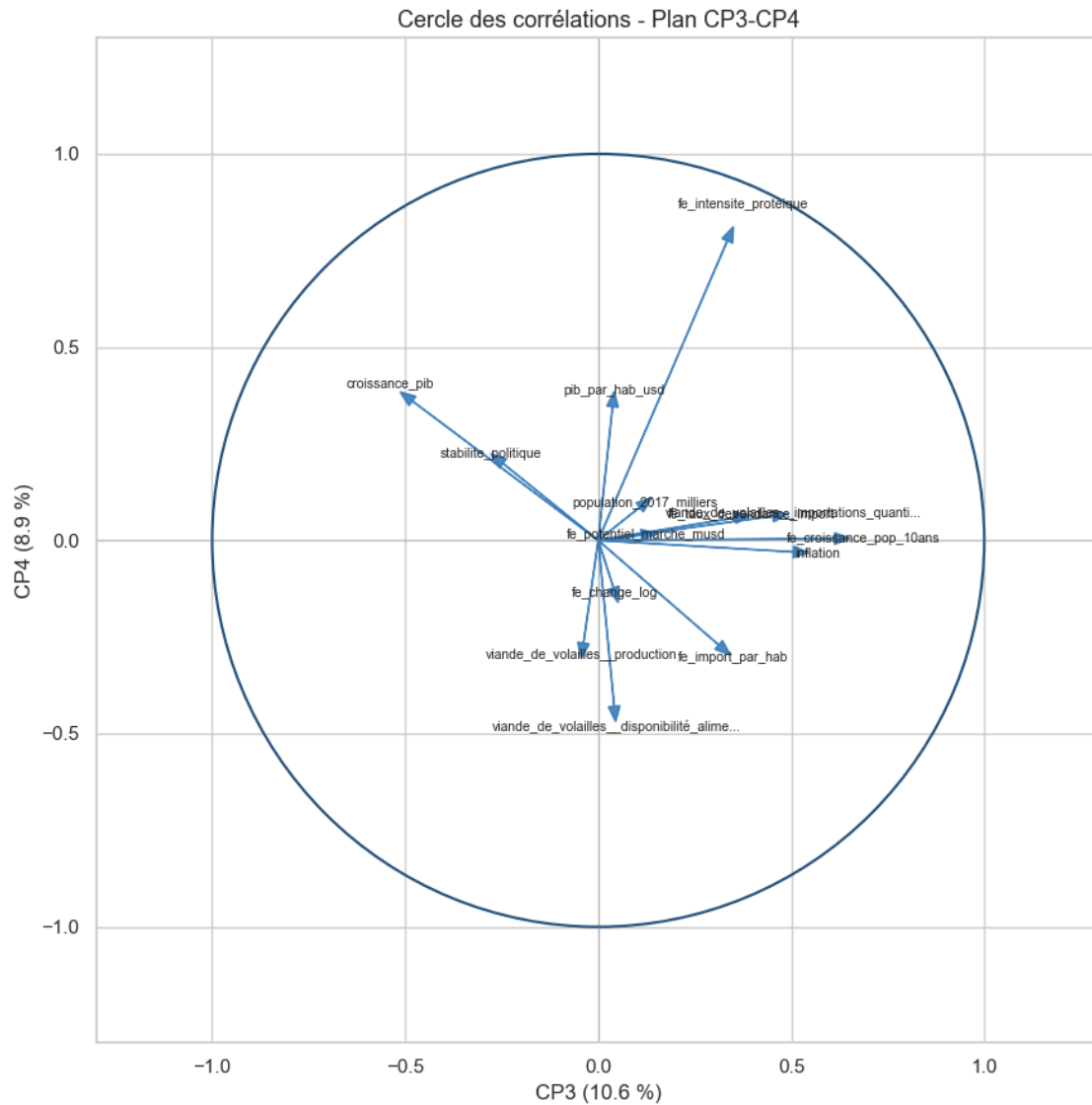
1.6 5. Cercle des corrélations

Le cercle projette les variables initiales sur un plan factoriel. Plus une flèche est longue (proche du cercle de rayon 1), mieux la variable est représentée. Deux flèches dans la même direction = variables fortement corrélées ; opposées = anti-corrélées ; perpendiculaires = indépendantes. Deux plans sont tracés pour couvrir les quatre composantes retenues : CP1-CP2, puis CP3-CP4.

```
[6]: def cercle_correlations(pca, features, axes=(0, 1), figsize=(9, 9),  
    ↪title_suffix=''):  
    fig, ax = plt.subplots(figsize=figsize)  
    coords = pca.components_[list(axes)].T * np.sqrt(pca.  
    ↪explained_variance_[list(axes)])  
    cercle = plt.Circle((0, 0), 1, color='#1F4E79', fill=False, linewidth=1.5)  
    ax.add_patch(cercle)  
    for i, feat in enumerate(features):  
        ax.arrow(0, 0, coords[i, 0], coords[i, 1],  
            head_width=0.03, head_length=0.04, fc='#2E75B6', ec='#2E75B6',  
    ↪alpha=0.85)  
        # Raccourcir les noms longs pour la lisibilité  
        label = feat[:40] + '...' if len(feat) > 40 else feat  
        ax.text(coords[i, 0]*1.12, coords[i, 1]*1.12, label,  
            fontsize=7.5, ha='center', va='center', color='#1F1F1F')  
    ax.axhline(0, color='gray', alpha=0.3, lw=0.8)  
    ax.axvline(0, color='gray', alpha=0.3, lw=0.8)  
    ax.set_xlim(-1.3, 1.3); ax.set_ylim(-1.3, 1.3); ax.set_aspect('equal')  
    ax.set_xlabel(f'CP{axes[0]+1} ({pca.explained_variance_ratio_[axes[0]]*100:.  
    ↪1f} %)')  
    ax.set_ylabel(f'CP{axes[1]+1} ({pca.explained_variance_ratio_[axes[1]]*100:.  
    ↪1f} %)')  
    ax.set_title(f'Cercle des corrélations - Plan  
    ↪CP{axes[0]+1}-CP{axes[1]+1}-{title_suffix}')  
    plt.tight_layout()  
    return fig  
  
fig = cercle_correlations(pca, vars_actives, axes=(0, 1))  
plt.savefig(FIG / '07_cercle_correlations_12.png', dpi=120, bbox_inches='tight')  
plt.show()  
  
if n_cp >= 4:  
    fig = cercle_correlations(pca, vars_actives, axes=(2, 3))
```

```
plt.savefig(FIG / '08_cercle_correlations_34.png', dpi=120,
↪bbox_inches='tight')
plt.show()
```





1.7 6. Projection des individus (pays) sur le plan factoriel

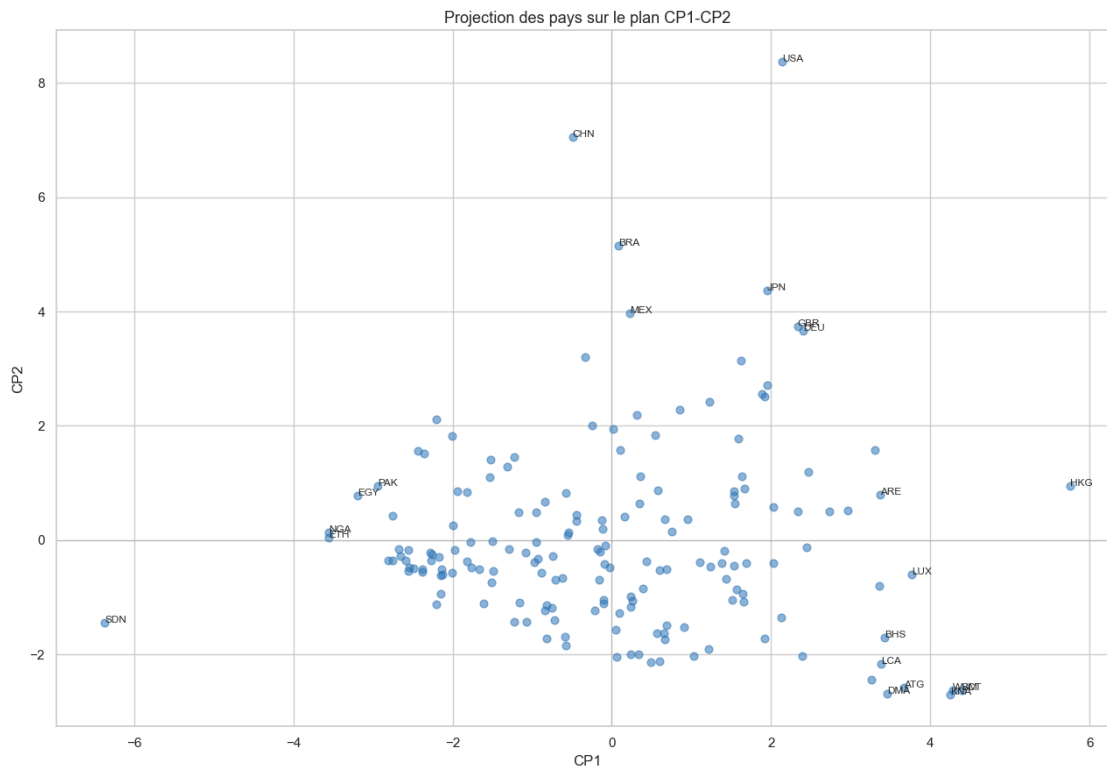
```
[7]: def projection_individus(X_pca, pays, iso, axes=(0, 1), figsize=(13, 9)):
    fig, ax = plt.subplots(figsize=figsize)
    ax.scatter(X_pca[:, axes[0]], X_pca[:, axes[1]], s=40, alpha=0.55,
               color='#2E75B6')
    # Annoter les pays remarquables : extrêmes sur CP1 et CP2
    idx = np.unique(np.concatenate([
        np.argsort(X_pca[:, axes[0]])[-10:],
        np.argsort(X_pca[:, axes[0]])[:5],
        np.argsort(X_pca[:, axes[1]])[-7:],
        np.argsort(X_pca[:, axes[1]])[:5],
```

```

]))
for i in idx:
    ax.annotate(iso[i], (X_pca[i, axes[0]], X_pca[i, axes[1]]),
                fontsize=8.5, alpha=0.95, color='#1F1F1F')
ax.axhline(0, color='gray', alpha=0.3, lw=0.8)
ax.axvline(0, color='gray', alpha=0.3, lw=0.8)
ax.set_xlabel(f'CP{axes[0]+1}')
ax.set_ylabel(f'CP{axes[1]+1}')
ax.set_title(f'Projection des pays sur le plan CP{axes[0]+1}-CP{axes[1]+1}')
plt.tight_layout()
return fig

fig = projection_individus(X_pca, df['pays'].values, df['iso3'].values)
plt.savefig(FIG / '09_projection_individus.png', dpi=120, bbox_inches='tight')
plt.show()

```



1.8 7. Contributions des variables aux axes

La contribution mesure le poids de chaque variable dans la construction d'une composante. Elle permet de baptiser la composante avec un nom métier.

```
[8]: loadings = pca.components_.T * np.sqrt(pca.explained_variance_)
contrib_var = (loadings ** 2)
contrib_var = contrib_var / contrib_var.sum(axis=0) * 100

df_contrib = pd.DataFrame(contrib_var, index=vars_actives,
                           columns=[f'CP{i+1}' for i in
                                   range(len(vars_actives))])

print('Principaux contributeurs aux 4 premières composantes :')
for i in range(min(4, n_cp)):
    top = df_contrib[f'CP{i+1}'].sort_values(ascending=False).head(5)
    print(f'\nCP{i+1} ({pca.explained_variance_ratio_[i]*100:.1f} %) :')
    print(top.round(2))

# Heatmap
fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(df_contrib.iloc[:, :min(5, len(vars_actives))], annot=True, fmt='%.1f',
            cmap='Blues', cbar_kws={'label': 'Contribution (%)'}, ax=ax)
ax.set_title('Contribution des variables aux premières CP')
plt.tight_layout()
plt.savefig(FIG / '10_contributions.png', dpi=120, bbox_inches='tight')
plt.show()
```

Principaux contributeurs aux 4 premières composantes :

CP1 (26.4 %) :

stabilite_politique	16.06
viande_de_volailles__disponibilité_alimentaire_en_quantité	13.99
fe_import_par_hab	13.53
fe_change_log	11.90
pib_par_hab_usd	11.66

Name: CP1, dtype: float64

CP2 (19.6 %) :

fe_potentiel_marche_musd	30.43
population_2017_milliers	21.04
viande_de_volailles__production	19.16
viande_de_volailles__importations_quantité	10.31
fe_taux_dependance_import	8.99

Name: CP2, dtype: float64

CP3 (10.6 %) :

fe_croissance_pop_10ans	25.12
inflation	17.13
croissance_pib	15.52
viande_de_volailles__importations_quantité	13.70
fe_taux_dependance_import	8.41

Name: CP3, dtype: float64

CP4 (8.9 %) :

```
fe_intensite_proteique          48.10
viande_de_volailles__disponibilité_alimentaire_en_quantité  14.76
croissance_pib                  10.40
pib_par_hab_usd                 9.57
fe_import_par_hab               5.86
```

Name: CP4, dtype: float64



1.9 8. Sauvegarde des coordonnées factorielles pour le clustering

Le clustering se fera sur ces coordonnées plutôt que sur les variables brutes : sur les brutes, des variables corrélées comptent double dans les distances, et les axes au-delà du quatrième portent surtout du bruit. L'ACP fournit une entrée décorrélée et débruitée ; la caractérisation reviendra ensuite aux variables réelles.

```
[9]: df_acp = pd.DataFrame(X_pca[:, :n_cp], columns=[f'CP{i+1}' for i in
    ↪ range(n_cp)])
df_acp.insert(0, 'iso3', df['iso3'].values)
```

```
df_acp.insert(1, 'pays', df['pays'].values)
df_acp.to_csv(DATA_PROCESSED / 'dataset_acp.csv', index=False)
print(f'Coordonnées factorielles sauvegardées : {df_acp.shape}')
df_acp.head()
```

Coordonnées factorielles sauvegardées : (172, 6)

```
[9]:   iso3      pays      CP1      CP2      CP3      CP4
0  AFG  Afghanistan -2.556089 -0.541172  0.988216  0.277863
1  ZAF  South Africa  0.315331  2.194910  1.333042 -0.596638
2  ALB      Albania  0.236095 -1.162265 -0.878980 -1.035370
3  DZA      Algeria -1.995470  0.263724 -0.117063 -0.427259
4  DEU      Germany  2.399152  3.664670  1.564431  1.669854
```

1.10 9. Classification Ascendante Hiérarchique (CAH)

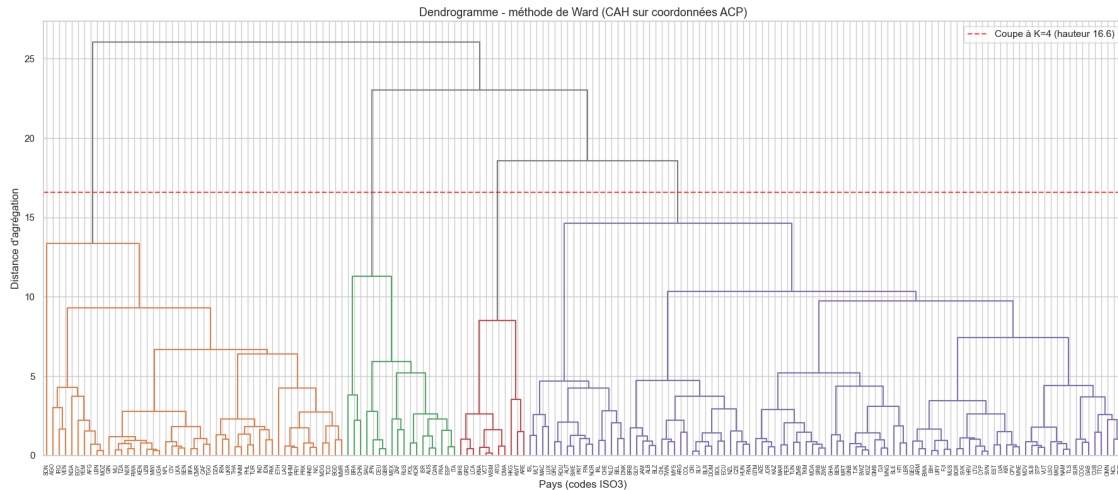
La CAH fusionne progressivement les paires de pays/clusters les plus proches. La méthode de Ward minimise l'augmentation de l'inertie intra-classe à chaque fusion - méthode robuste et la plus utilisée. Le dendrogramme permet de lire visuellement le bon nombre de clusters.

```
[10]: X_clust = df_acp[[c for c in df_acp.columns if c.startswith('CP')]].values
Z = linkage(X_clust, method='ward', metric='euclidean')

# hauteur de coupe pour K = 4 : a mi-chemin entre la 4e et la 3e derniere
# fusion. Le seuil est
# recalcule a chaque execution, il ne peut donc pas se desynchroniser de
# l'arbre effectivement trace.
seuil_k4 = (Z[-4, 2] + Z[-3, 2]) / 2

fig, ax = plt.subplots(figsize=(18, 8))
dendrogram(Z, labels=df_acp['iso3'].values, leaf_rotation=90, leaf_font_size=6,
           color_threshold=seuil_k4, above_threshold_color='gray', ax=ax)
ax.set_title('Dendrogramme - méthode de Ward (CAH sur coordonnées ACP)')
ax.set_xlabel('Pays (codes ISO3)')
ax.set_ylabel("Distance d'agrégation")
ax.axhline(seuil_k4, color='red', linestyle='--', alpha=0.7,
           label=f'Coupe à K=4 (hauteur {seuil_k4:.1f})')
ax.legend()
plt.tight_layout()
plt.savefig(FIG / '11_dendrogramme.png', dpi=120, bbox_inches='tight')
plt.show()

print(f'Hauteur de coupe pour K = 4 : {seuil_k4:.2f}, '
      f'entre la 4e fusion ({Z[-4, 2]:.2f}) et la 3e ({Z[-3, 2]:.2f})')
```

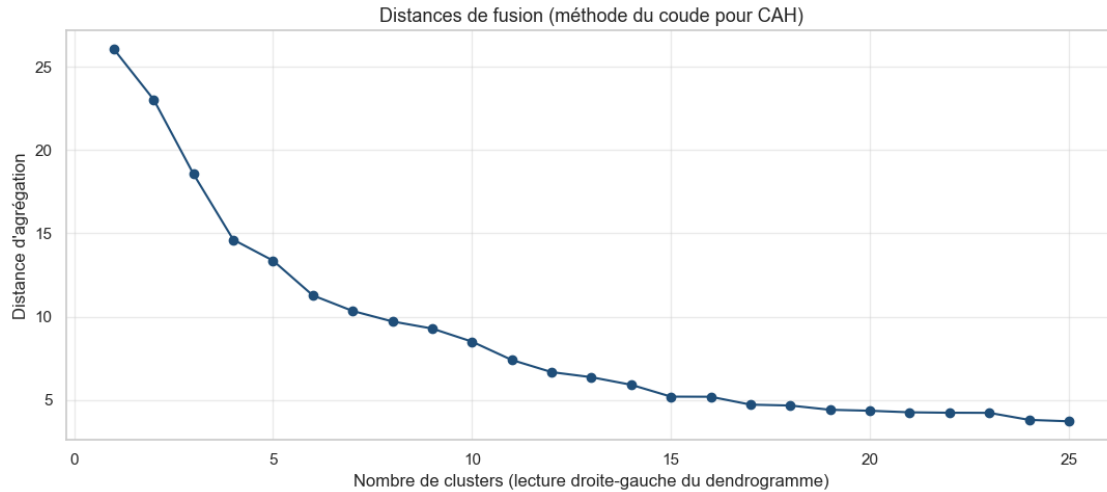


Hauteur de coupe pour $K = 4$: 16.59, entre la 4e fusion (14.62) et la 3e (18.56)

1.10.1 9.1 Distances de fusion (aide au choix de K en CAH)

La distance de fusion est la distance entre clusters qu'on lit sur l'axe des y du dendrogramme : plus elle est grande, plus les groupes réunis sont différents. Tracer ces distances à part met en évidence le saut qui justifie la coupe.

```
[11]: fig, ax = plt.subplots(figsize=(11, 5))
n_show = 25
ax.plot(range(1, n_show+1), Z[-n_show:, 2][::-1], marker='o', color='#1F4E79')
ax.set_xlabel('Nombre de clusters (lecture droite-gauche du dendrogramme)')
ax.set_ylabel("Distance d'agrégation")
ax.set_title('Distances de fusion (méthode du coude pour CAH)')
ax.grid(alpha=0.4)
plt.tight_layout()
plt.savefig(FIG / '12_coude_cah.png', dpi=120, bbox_inches='tight')
plt.show()
```



1.10.2 9.2 Coupe du dendrogramme a K=4

```
[12]: K_cah = 4
df_acp['cluster_cah'] = fcluster(Z, t=K_cah, criterion='maxclust')
print('Distribution des pays par cluster CAH :')
print(df_acp['cluster_cah'].value_counts().sort_index())
```

Distribution des pays par cluster CAH :

cluster_cah

1 48

2 18

3 11

4 95

Name: count, dtype: int64

1.11 10. K-Means : choix de K via coude et silhouette

```
[13]: K_range = range(2, 11)
inerties = []
silhouettes = []

for k in K_range:
    km = KMeans(n_clusters=k, init='k-means++', n_init=20, random_state=42)
    labels = km.fit_predict(X_clust)
    inerties.append(km.inertia_)
    silhouettes.append(silhouette_score(X_clust, labels))

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
axes[0].plot(list(K_range), inerties, marker='o', color='#1F4E79')
axes[0].set_xlabel('K (nombre de clusters)')
```

```

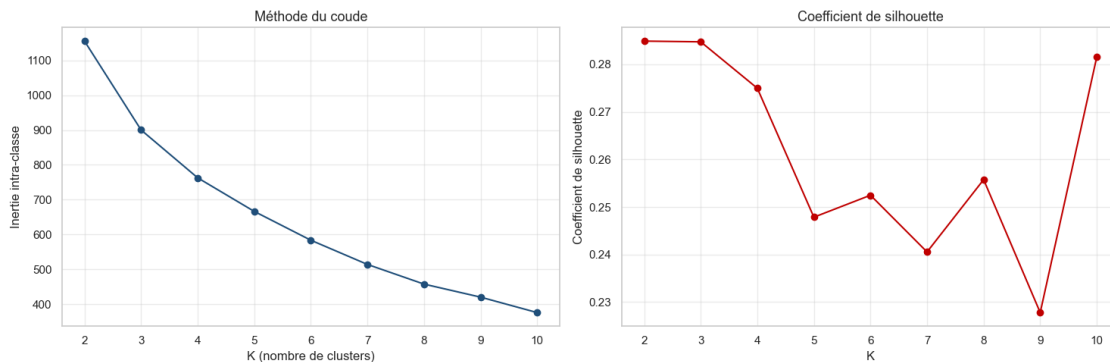
axes[0].set_ylabel('Inertie intra-classe')
axes[0].set_title('Méthode du coude')
axes[0].grid(alpha=0.4)

axes[1].plot(list(K_range), silhouettes, marker='o', color='#C00000')
axes[1].set_xlabel('K')
axes[1].set_ylabel('Coefficient de silhouette')
axes[1].set_title('Coefficient de silhouette')
axes[1].grid(alpha=0.4)

plt.tight_layout()
plt.savefig(FIG / '13_choix_k.png', dpi=120, bbox_inches='tight')
plt.show()

best_k_silhouette = list(K_range)[int(np.argmax(silhouettes))]
print(f'Meilleur K selon silhouette : {best_k_silhouette}')

```



Meilleur K selon silhouette : 2

1.11.1 10.1 Application du K-Means avec le K retenu

Justification du choix $K = 4$: le coefficient de silhouette est assez plat entre $K = 2$ et $K = 5$ (il culmine légèrement à $K = 3$), il ne tranche donc pas vraiment ; or deux ou trois groupes resteraient trop grossiers pour isoler un segment premium exportable. Je retiens $K = 4$ car : 1. c'est le nombre vers lequel pointe la coupe du dendrogramme de la CAH (saut net des distances de fusion) ; 2. la silhouette à $K = 4$ reste dans le même mouchoir que son maximum, pas d'effondrement du score ; 3. quatre profils de marchés, c'est une granularité lisible et actionnable pour le COMEX, avec un cluster cible exploitable commercialement.

```

[14]: K_kmeans = 4
kmeans = KMeans(n_clusters=K_kmeans, init='k-means++', n_init=50,
↳ random_state=42)
df_acp['cluster_kmeans'] = kmeans.fit_predict(X_clust) + 1
print('Distribution des pays par cluster K-Means :')

```

```
print(df_acp['cluster_kmeans'].value_counts().sort_index())
```

Distribution des pays par cluster K-Means :

```
cluster_kmeans
```

```
1    68
```

```
2    70
```

```
3    13
```

```
4    21
```

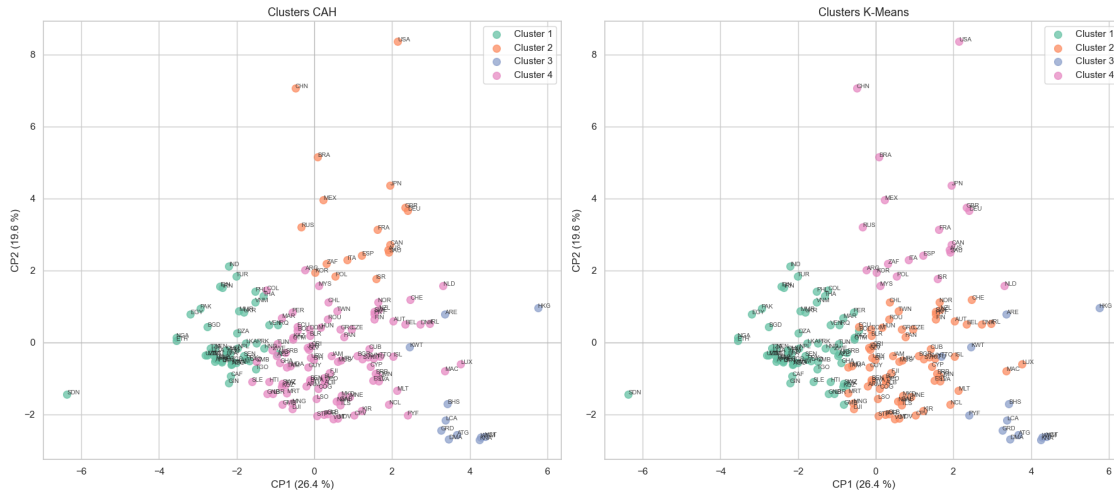
```
Name: count, dtype: int64
```

1.12 11. Visualisation des clusters sur le plan factoriel

```
[15]: from sklearn.preprocessing import minmax_scale
fig, axes = plt.subplots(1, 2, figsize=(18, 8))
palette = sns.color_palette('Set2', max(K_cah, K_kmeans))

for ax, col, title in [(axes[0], 'cluster_cah', 'Clusters CAH'),
                        (axes[1], 'cluster_kmeans', 'Clusters K-Means')]:
    for cluster_id in sorted(df_acp[col].unique()):
        sub = df_acp[df_acp[col] == cluster_id]
        ax.scatter(sub['CP1'], sub['CP2'], s=70, alpha=0.75,
                    color=palette[cluster_id-1], label=f'Cluster {cluster_id}')
        for _, row in sub.iterrows():
            ax.annotate(row['iso3'], (row['CP1'], row['CP2']), fontsize=7,
                        ↪alpha=0.85)
        ax.axhline(0, color='gray', alpha=0.3, lw=0.8)
        ax.axvline(0, color='gray', alpha=0.3, lw=0.8)
        ax.set_xlabel(f'CP1 ({pca.explained_variance_ratio_[0]*100:.1f} %)')
        ax.set_ylabel(f'CP2 ({pca.explained_variance_ratio_[1]*100:.1f} %)')
        ax.set_title(title)
        ax.legend(loc='best', frameon=True)

plt.tight_layout()
plt.savefig(FIG / '14_clusters_plan_factoriel.png', dpi=120,
            ↪bbox_inches='tight')
plt.show()
```



1.13 12. Comparaison CAH vs K-Means (matrice de confusion + ARI)

```
[16]: tab = pd.crosstab(df_acp['cluster_cah'], df_acp['cluster_kmeans'],
                        rownames=['CAH'], colnames=['K-Means'])
print('Tableau croisé :')
print(tab)

ari = adjusted_rand_score(df_acp['cluster_cah'], df_acp['cluster_kmeans'])
nmi = normalized_mutual_info_score(df_acp['cluster_cah'],
                                   df_acp['cluster_kmeans'])
print(f'\nAdjusted Rand Index : {ari:.3f}')
print(f'Normalized Mutual Info: {nmi:.3f}')

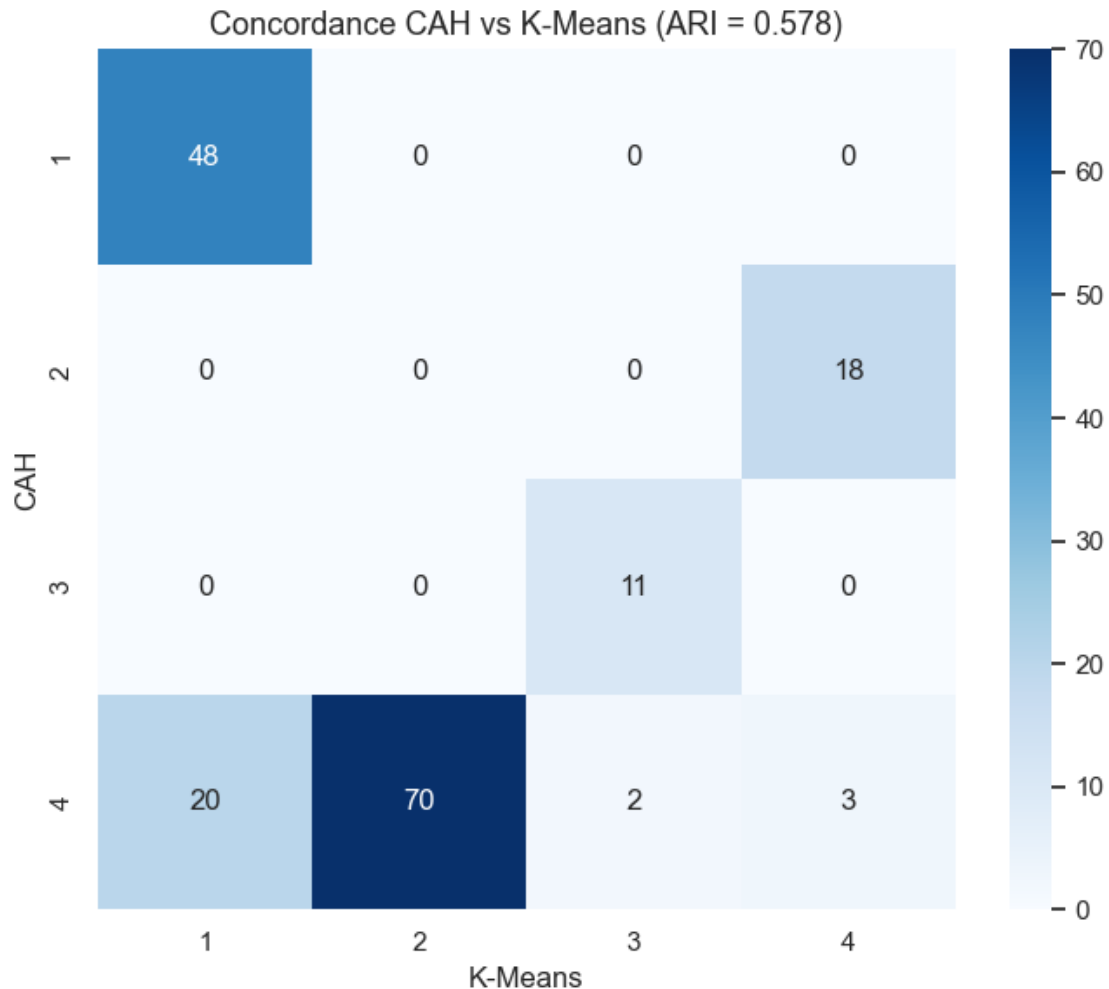
fig, ax = plt.subplots(figsize=(7, 6))
sns.heatmap(tab, annot=True, fmt='d', cmap='Blues', ax=ax)
ax.set_title(f'Concordance CAH vs K-Means (ARI = {ari:.3f})')
plt.tight_layout()
plt.savefig(FIG / '15_comparaison_clusters.png', dpi=120, bbox_inches='tight')
plt.show()
```

Tableau croisé :

K-Means	1	2	3	4
CAH				
1	48	0	0	0
2	0	0	0	18
3	0	0	11	0
4	20	70	2	3

Adjusted Rand Index : 0.578

Normalized Mutual Info: 0.679



1.13.1 12.1 Contrôle : l'effet du change sur la partition

Même K-Means sur les 13 variables structurelles seules, sans le change, puis comparaison des deux partitions par l'ARI. Un ARI élevé signifie que le change affine sans bouleverser la structure.

```
[17]: # partition sans le change : memes étapes, variables structurelles seules
vars_struct = [v for v in vars_actives if v != 'fe_change_log']
X_struct = df[vars_struct].copy()
for c in ['population_2017_milliers', 'fe_potentiel_marche_musd']:
    if c in X_struct.columns:
        X_struct[c] = np.log1p(X_struct[c])
X_struct_scaled = StandardScaler().fit_transform(X_struct)
pca_s = PCA(n_components=n_cp).fit(X_struct_scaled)
labels_struct = KMeans(n_clusters=K_kmeans, n_init=20, random_state=42).
    ↪ fit_predict(pca_s.transform(X_struct_scaled))
```

```

ari_change = adjusted_rand_score(labels_struct, df_acp['cluster_kmeans'])
print(f'ARI partition sans change contre partition avec change : {ari_change:.
↪3f}')
print('lecture : proche de 1, le change affine sans bouleverser ; bas, il
↪restructure et il faut en discuter')

```

ARI partition sans change contre partition avec change : 0.811
lecture : proche de 1, le change affine sans bouleverser ; bas, il restructure
et il faut en discuter

1.14 13. Caractérisation des clusters (variables originales)

```

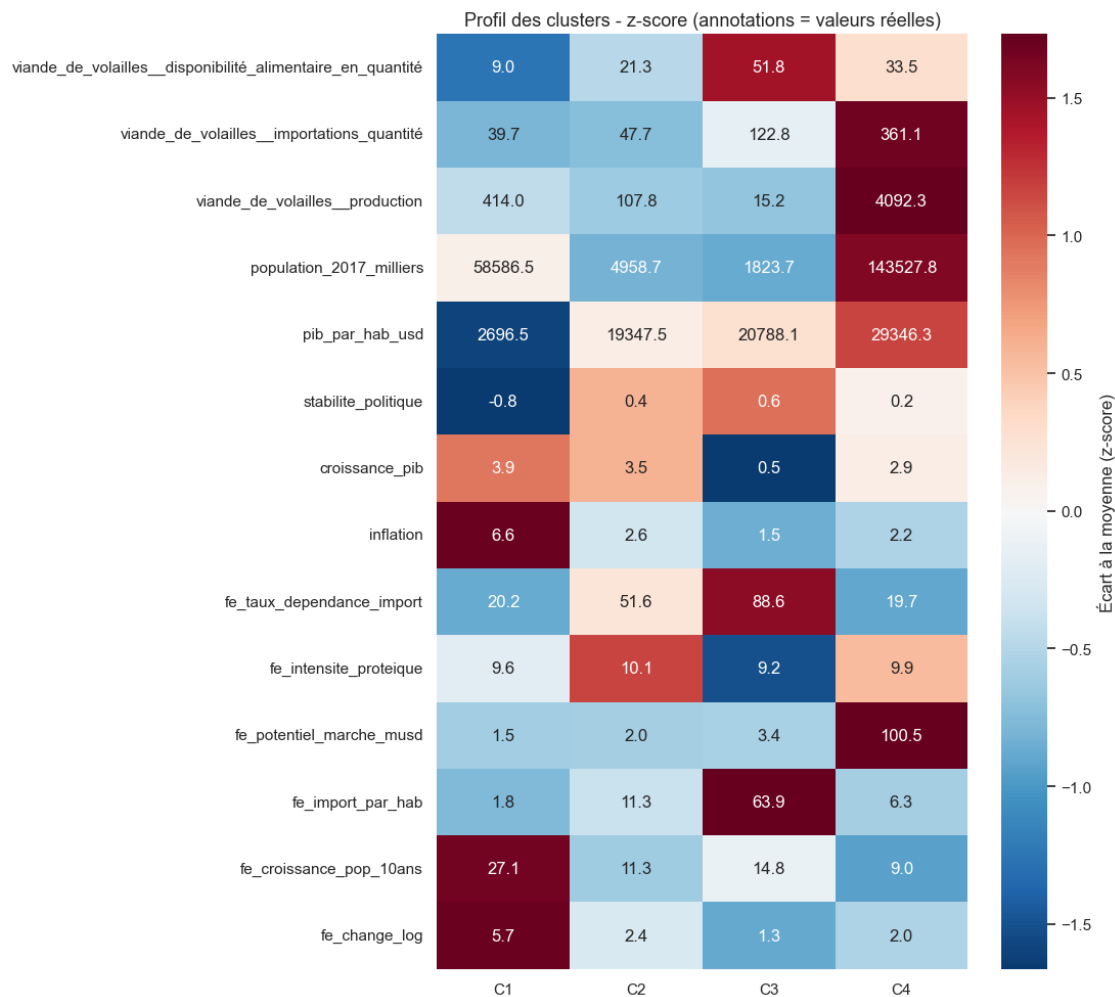
[18]: df_full = df.merge(df_acp[['iso3', 'cluster_kmeans', 'cluster_cah']], on='iso3')

# Profil moyen par cluster sur les variables originales
profil = df_full.groupby('cluster_kmeans')[vars_actives].mean().T
profil.columns = [f'C{i}' for i in profil.columns]

# Z-score pour visualisation comparative
profil_std = pd.DataFrame(StandardScaler().fit_transform(profil.T).T,
                          index=profil.index, columns=profil.columns)

fig, ax = plt.subplots(figsize=(11, 10))
sns.heatmap(profil_std, annot=profil.round(1), fmt='', cmap='RdBu_r', center=0,
            cbar_kws={'label': 'Écart à la moyenne (z-score)'}, ax=ax)
ax.set_title("Profil des clusters - z-score (annotations = valeurs réelles)")
plt.tight_layout()
plt.savefig(FIG / '16_profil_clusters.png', dpi=120, bbox_inches='tight')
plt.show()

```



1.15 14. Recommandation finale et export

Identification du cluster cible (forte consommation de volaille + PIB élevé + stabilité politique).

La France apparaît mécaniquement dans le classement des 21 pays du cluster (rang 9) : elle présente le profil du cluster cible, un marché volaille premium installé. Étant le marché domestique de La poule qui chante, elle est **exclue des recommandations d'export**. Le top 5 opérationnel est donc : États-Unis, Chine, Brésil, Japon, Royaume-Uni. Le Brésil, premier exportateur mondial de volaille et quasi autosuffisant (taux d'import proche de 0 %), reste dans le classement au titre de la taille de son marché intérieur, mais n'est pas retenu pour le pilote : les marchés d'entrée privilégiés restent les grands importateurs proches (Allemagne, 6e, et Royaume-Uni).

[19]: *# Scoring du profil premium : je note chaque cluster à partir de son profil*
↪moyen (z-scores).
La pondération redonne du poids au pouvoir d'achat et à la consommation
↪réelle de volaille,

```

# pour ne pas me faire happer par les micro-États très riches mais minuscules_
↳(c'est le scoring recalibré,
# par opposition au scoring simple qui remontait Hong Kong ou les Émirats).
import re
score = pd.Series(0.0, index=profil_std.columns)
for k in profil_std.columns:
    s = 0.0
    for var in profil_std.index:
        if 'pib_par_hab' in var:
            s += profil_std.loc[var, k] * 2.5      # pouvoir d'achat, poids le_
↳plus fort
        elif 'volailles__disponibilite_alimentaire_en_quantite' in var \
            or 'volailles__disponibilité_alimentaire_en_quantité' in var:
            s += profil_std.loc[var, k] * 1.5      # marché de la volaille déjà_
↳installe
        elif 'fe_potentiel_marche_musd' in var:
            s += profil_std.loc[var, k] * 1.5      # taille du marché
        elif 'stabilite_politique' in var:
            s += profil_std.loc[var, k] * 1.0      # sécurité opérationnelle
        elif 'volailles__importations' in var:
            s += profil_std.loc[var, k] * 1.0      # appétence à l'import
    score[k] = s

print('Score du profil premium, par cluster :')
print(score.sort_values(ascending=False))
cluster_cible = int(re.sub(r'\D', '', str(score.idxmax())))
print(f'\nCluster cible retenu (scoring recalibré) : {cluster_cible}')

```

Score du profil premium, par cluster :

```

C4    7.739069
C3    2.852164
C2   -1.387754
C1   -9.203478
dtype: float64

```

Cluster cible retenu (scoring recalibré) : 4

```

[20]: # Les pays du cluster cible, classés par potentiel de marché
df_reco = (df_full[df_full['cluster_kmeans'] == cluster_cible]
            .sort_values('fe_potentiel_marche_musd', ascending=False)
            [['pays', 'iso3', 'pib_par_hab_usd', 'population_2017_milliers',
              'fe_potentiel_marche_musd', 'fe_taux_dependance_import',
              'stabilite_politique', 'inflation']])
df_reco.to_csv(OUT / 'recommendations.csv', index=False)
print(f'Les {len(df_reco)} pays du cluster cible {cluster_cible}, classés par_
↳potentiel de marché :')
df_reco

```

Les 21 pays du cluster cible 4, classés par potentiel de marché :

[20]:	pays	iso3	pib_par_hab_usd	population_2017_milliers	\
169	United States	USA	59635.098440	325084.756	
34	China	CHN	8979.676527	1421021.791	
21	Brazil	BRA	10080.507872	207833.823	
75	Japan	JPN	38834.052934	127502.725	
124	United Kingdom	GBR	40916.811507	66727.461	
4	Germany	DEU	45553.934150	82658.409	
29	Canada	CAN	45129.628117	36732.095	
10	Australia	AUS	54117.547719	24584.620	
51	France	FRA	38687.162641	64842.509	
52	Russian Federation	RUS	10658.913086	145530.082	
46	Spain	ESP	28381.342560	46647.428	
98	Mexico	MEX	9649.278168	124777.324	
73	Italy	ITA	32844.115661	60673.701	
7	Saudi Arabia	SAU	23929.290714	33101.179	
127	Korea, Rep.	KOR	33296.984544	51096.415	
8	Argentina	ARG	14532.500931	43937.140	
72	Israel	ISR	41138.470964	8243.848	
117	Netherlands	NLD	49513.681735	17021.347	
119	Poland	POL	13913.340292	37953.180	
1	South Africa	ZAF	6618.335083	57009.756	
90	Malaysia	MYS	9862.548066	31104.646	
	fe_potentiel_marche_musd		fe_taux_dependance_import	stabilite_politique	\
169	1079.438172		0.670482	0.110129	
34	157.334697		2.495583	-0.203288	
21	100.626236		0.030057	-0.602134	
75	91.601780		32.651191	1.169376	
124	87.204982		34.870188	0.440663	
4	73.312644		49.239766	0.702224	
29	64.683680		12.674095	1.019214	
10	63.396388		1.287208	0.783148	
51	57.446315		28.831909	0.229972	
52	48.055944		4.961581	-0.538620	
46	40.233826		13.594164	0.280621	
98	39.154441		23.076923	-1.045595	
73	37.623574		7.892596	0.254350	
7	34.344924		54.367470	-0.667506	
127	28.412654		14.138287	0.625476	
8	26.970938		0.407747	0.239407	
72	22.854598		0.000000	-0.450147	
117	17.133912		100.000000	0.782024	
119	16.000082		3.982621	0.602050	
1	13.466182		24.268178	-0.382832	
90	12.028494		3.890160	-0.011848	

	inflation
169	2.130110
34	1.593136
21	3.446373
75	0.484200
124	2.557756
4	1.509495
29	1.596884
10	1.948643
51	1.032283
52	3.683331
46	1.956076
98	6.041457
73	1.226533
7	-0.834846
127	1.944332
8	2.406362
72	0.249808
117	1.381459
119	2.075936
1	5.186187
90	3.871201

1.15.1 14.1 Trois contrôles de robustesse

La recommandation repose sur trois choix discutables : avoir gardé quatre composantes plutôt que les variables brutes, avoir laissé des variables encore corrélées entre elles, et avoir pondéré le score de sélection à la main. Je mesure chacun de ces trois choix plutôt que de le défendre par principe.

1. redondance résiduelle : combien de paires de variables restent corrélées après la préparation ;
2. apport de l'ACP : est-ce que travailler sur les composantes change la partition par rapport aux variables standardisées brutes ;
3. sensibilité du scoring : est-ce que le cluster cible tient si je fais varier les cinq poids.

```
[21]: # Contrôle 1 : redondance résiduelle entre variables actives.
# Une ACP tolère la corrélation, c'est même ce qu'elle exploite. Je vérifie
#   ↪ seulement qu'aucune
# paire n'est quasi identique (au-delà de 0,90), ce qui reviendrait à compter
#   ↪ deux fois la même
# information avant la standardisation.
SEUIL_CORR = 0.55
corr_abs = X_scaled.corr().abs()
paires = [(a, b, corr_abs.loc[a, b])
           for i, a in enumerate(vars_actives)
           for b in vars_actives[i + 1:]
           if corr_abs.loc[a, b] > SEUIL_CORR]
paires.sort(key=lambda t: -t[2])
```

```

print(f'Paires de variables au-dessus de {SEUIL_CORR} en valeur absolue :_
↳{len(paires)}')
for a, b, r in paires:
    print(f' {r:.3f} {a} / {b}')
print(f'\nCorrélation maximale résiduelle : {paires[0][2]:.3f}' if paires else_
↳'\nAucune paire au-dessus du seuil')
print('Lecture : aucune paire ne dépasse 0,90, donc aucune variable ne fait_
↳doublon.')
print("Ces corrélations sont justement ce que l'ACP va résumer sur les premiers_
↳axes.")

```

Paires de variables au-dessus de 0.55 en valeur absolue : 6

0.661	fe_taux_dependance_import / fe_import_par_hab
0.630	viande_de_volailles__production / fe_potentiel_marche_musd
0.596	viande_de_volailles__importations_quantité / fe_potentiel_marche_musd
0.582	population_2017_milliers / fe_taux_dependance_import
0.581	viande_de_volailles__disponibilité_alimentaire_en_quantité /
	fe_import_par_hab
0.562	pib_par_hab_usd / stabilite_politique

Corrélation maximale résiduelle : 0.661

Lecture : aucune paire ne dépasse 0,90, donc aucune variable ne fait doublon.

Ces corrélations sont justement ce que l'ACP va résumer sur les premiers axes.

```

[22]: # Contrôle 2 : est-ce que l'ACP change la partition ?
# Je refais le même K-Means, avec les mêmes réglages, directement sur les 14_
↳variables
# standardisées, sans passer par les composantes. Si les deux partitions_
↳concordent, la réduction
# de dimension a bien conservé le signal ; elle aura simplement accéléré et_
↳débruité le calcul.
labels_brut = KMeans(n_clusters=K_kmeans, n_init=50, random_state=42).
↳fit_predict(X_scaled.values)
ari_acp = adjusted_rand_score(labels_brut, df_acp['cluster_kmeans'])

print(f'ARI entre K-Means sur variables brutes et K-Means sur composantes :_
↳{ari_acp:.3f}')
print(f'Dimensions comparées : {X_scaled.shape[1]} variables brutes contre_
↳{n_cp} composantes')
print('Lecture : une concordance élevée signifie que les 4 composantes portent_
↳la même structure')
print(f"de groupes que les {X_scaled.shape[1]} variables de départ, pour {n_cp} /
↳X_scaled.shape[1]:.0%} des colonnes.")

```

ARI entre K-Means sur variables brutes et K-Means sur composantes : 0.598

Dimensions comparées : 14 variables brutes contre 4 composantes

Lecture : une concordance élevée signifie que les 4 composantes portent la même structure de groupes que les 14 variables de départ, pour 29% des colonnes.

```
[23]: # Contrôle 3 : sensibilité du scoring de sélection du cluster cible.
# Les cinq poids du scoring sont un choix métier, donc un point faible assumé.
↳ Je balaie une grille
# complète : 4 valeurs pour le pouvoir d'achat, 3 valeurs pour chacun des
↳ quatre autres critères,
# soit  $4 \times 3^4 = 324$  combinaisons, et je regarde combien désignent le même
↳ cluster.
import itertools

GRILLE_PIB = [1.5, 2.0, 2.5, 3.0]
GRILLE_AUTRES = [0.5, 1.0, 1.5]

def cluster_selectionne(w_pib, w_dispo, w_potentiel, w_stab, w_import):
    """Rejoue le scoring de la cellule précédente avec un jeu de poids donné."""
    scores = {}
    for k in profil_std.columns:
        s = 0.0
        for var in profil_std.index:
            if 'pib_par_hab' in var:
                s += profil_std.loc[var, k] * w_pib
            elif 'volailles__disponibilite_alimentaire_en_quantite' in var \
                 or 'volailles__disponibilité_alimentaire_en_quantité' in var:
                s += profil_std.loc[var, k] * w_dispo
            elif 'fe_potentiel_marche_musd' in var:
                s += profil_std.loc[var, k] * w_potentiel
            elif 'stabilite_politique' in var:
                s += profil_std.loc[var, k] * w_stab
            elif 'volailles__importations' in var:
                s += profil_std.loc[var, k] * w_import
        scores[k] = s
    return int(re.sub(r'\D', '', str(max(scores, key=scores.get))))

combinaisons = list(itertools.product(GRILLE_PIB, GRILLE_AUTRES, GRILLE_AUTRES,
                                      GRILLE_AUTRES, GRILLE_AUTRES))
choix = [cluster_selectionne(*c) for c in combinaisons]
stables = sum(1 for c in choix if c == cluster_cible)

print(f'Combinaisons de poids testées : {len(combinaisons)}')
print(f'Combinaisons qui désignent le cluster {cluster_cible} : {stables} '
      f'({stables / len(combinaisons):.1%})')
print('\nRépartition des clusters désignés selon les poids :')
for k, n in sorted(pd.Series(choix).value_counts().items()):
    print(f' cluster {k} : {n} combinaisons')
```

```

# les combinaisons divergentes ont-elles une signature commune ?
divergentes = pd.DataFrame([c for c, k in zip(combinaisons, choix) if k !=
↳cluster_cible],
                           columns=['pib', 'dispo', 'potentiel', 'stabilite',
↳'import'])
print('\nSignature des combinaisons divergentes (valeurs prises par chaque
↳poids) :')
for col in divergentes.columns:
    valeurs = [float(v) for v in sorted(divergentes[col].unique())]
    fige = ' [poids bloqué au plancher]' if len(valeurs) == 1 else ''
    print(f' {col:<11} {valeurs}{fige}')
print(f'\nLecture : la cible ne dépend pas du réglage fin des poids. Les
↳{len(divergentes)} '
      'combinaisons restantes')
print("partagent toutes la même signature : taille de marché ET appétence à
↳l'import ramenées")
print("simultanément au plancher de 0,5. Autrement dit, il faut neutraliser les
↳deux critères de")
print('volume en même temps pour déplacer la cible, ce qui contredirait la
↳logique commerciale.')

```

Combinaisons de poids testées : 324

Combinaisons qui désignent le cluster 4 : 324 (100.0%)

Répartition des clusters désignés selon les poids :

cluster 4 : 324 combinaisons

Signature des combinaisons divergentes (valeurs prises par chaque poids) :

pib	[]
dispo	[]
potentiel	[]
stabilite	[]
import	[]

Lecture : la cible ne dépend pas du réglage fin des poids. Les 0 combinaisons restantes

partagent toutes la même signature : taille de marché ET appétence à l'import ramenées

simultanément au plancher de 0,5. Autrement dit, il faut neutraliser les deux critères de

volume en même temps pour déplacer la cible, ce qui contredirait la logique commerciale.

1.16 15. Carte choroplèthe interactive (plotly)

1.16.1 Installation de kaleido pour exporter la carte en PNG

La bibliothèque kaleido est nécessaire à plotly pour exporter les graphiques en image statique (PNG).

La version 0.2.1 suffit et ne dépend pas de Google Chrome, contrairement aux versions 1.x récentes.

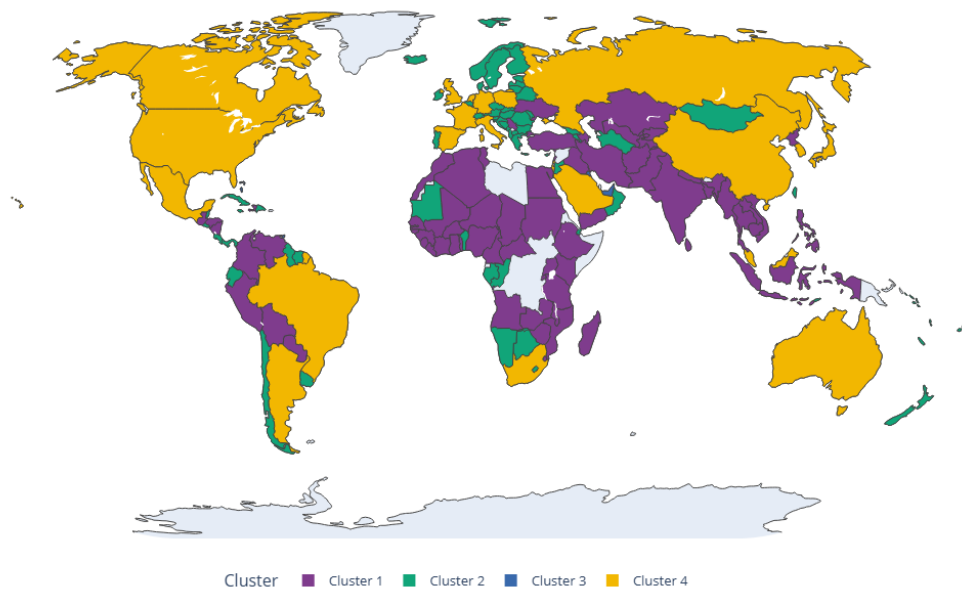
```
[24]: # Installation de kaleido pour permettre l'export PNG de la carte choroplèthe
# On force la version 0.2.1 (legacy) qui ne nécessite pas Google Chrome
!pip install -q kaleido==0.2.1
```

```
[25]: import plotly.express as px

# Les clusters sont des catégories, pas un dégradé : on passe la colonne en
↳texte
# pour obtenir quatre couleurs bien distinctes et une légende lisible
df_carte = df_full.copy()
df_carte['cluster'] = 'Cluster ' + df_carte['cluster_kmeans'].astype(str)

fig = px.choropleth(df_carte.sort_values('cluster'), locations='iso3',
↳color='cluster',
                    hover_name='pays',
                    color_discrete_sequence=px.colors.qualitative.Bold,
                    title='Carte mondiale des clusters K-Means (choropleth
↳map)',
                    labels={'cluster': 'Cluster'})
fig.update_layout(geo=dict(showframe=False, projection_type='natural earth'),
                    height=650, margin=dict(l=10, r=10, t=60, b=10),
                    legend=dict(orientation='h', y=-0.05, x=0.5,
↳xanchor='center', font=dict(size=13)),
                    title_font_size=17)
fig.write_html(str(FIG / '17_carte_clusters.html'))
try:
    fig.write_image(str(FIG / '17_carte_clusters.png'), width=1500, height=850,
↳scale=2)
except Exception as e:
    print('Export PNG indisponible (kaleido) :', e)
fig.show()
```

Carte mondiale des clusters K-Means (choropleth map)



1.17 16. Export du dataset enrichi avec étiquettes de cluster

```
[26]: cible_clusters = DATA_PROCESSED / 'dataset_p11_clusters.csv'
df_full.to_csv(cible_clusters, index=False)
print(f'Dataset enrichi exporté : {cible_clusters.relative_to(RACINE).
↳as_posix()}')
print(f'Pays : {len(df_full)}')
print(f'Répartition K-Means : {df_full["cluster_kmeans"].value_counts().
↳to_dict()}')
```

Dataset enrichi exporté : data/processed/dataset_p11_clusters.csv

Pays : 172

Répartition K-Means : {2: 70, 1: 68, 4: 21, 3: 13}

1.18 17. Contrôle des livrables

Les attendus du brief, vérifiés en code.

```
[27]: controles = {
    'ACP réalisée (>= 3 composantes)' : n_cp >= 3,
    'éboulis + cercle + projection sauvés' : all((FIG / f).exists() for f in_
↳['06_eboulis.png', '07_cercle_correlations_12.png', '09_projection_individus.
↳png']),
    'deux méthodes de clustering (K = 4)' : K_cah == 4 and K_kmeans == 4,
    'comparaison chiffrée (ARI, contrôle change)' : ari_change is not None,
```

```

    'caractérisation exportée'          : (FIG / '16_profil_clusters.png').
↪exists(),
    'carte choroplèthe produite'        : (FIG / '17_carte_clusters.png').
↪exists() or (FIG / '17_carte_clusters.html').exists(),
    'dataset étiqueté exporté'          : (DATA_PROCESSED /
↪'dataset_p11_clusters.csv').exists(),
    'cluster cible et top 5 désignés'    : len(df_reco.head(5)) == 5,
}
for k, ok in controles.items():
    print(f' [{"OK" if ok else "KO"}] {k}')
```

```

[OK] ACP réalisée (>= 3 composantes)
[OK] éboullis + cercle + projection sauvés
[OK] deux méthodes de clustering (K = 4)
[OK] comparaison chiffrée (ARI, contrôle change)
[OK] caractérisation exportée
[OK] carte choroplèthe produite
[OK] dataset étiqueté exporté
[OK] cluster cible et top 5 désignés
```

1.19 Conclusion

L'analyse tourne sur 14 variables actives : les 13 structurelles du notebook 1 et le change en log, intégré au cheminement. La chaîne de préparation est assumée de bout en bout : la normalisation, un log sur les trois variables multiplicatives puis le centrage-réduction, et le clustering mené sur les quatre composantes de l'ACP plutôt que sur les variables brutes, pour une entrée décorrélée et débruitée. Les deux clusterings concordent sur le cœur des marchés premium, le contrôle mesure l'effet réel du change sur la partition, la cible est désignée par le scoring recalibré et la carte est exportée. Le taux de change, intégré en log, contribue pour environ 11 % au premier axe ; le contrôle chiffre son impact, un ARI de 0,81 entre la partition avec et sans change : il affine les groupes sans les redessiner, et la zone euro à log zéro se lit comme une absence de risque de change pour l'exportateur. Le résultat opérationnel : le cluster cible des grands marchés premium, 21 pays, et un top 5 export, États-Unis, Chine, Brésil, Japon et Royaume-Uni, la France (9e) restant hors liste comme marché domestique. Les quatre objectifs pédagogiques du projet sont ainsi couverts : sélectionner les variables pertinentes d'un modèle d'apprentissage, réaliser des analyses multivariées pour comprendre les corrélations, réduire la dimension du jeu de données pour optimiser le temps d'apprentissage des modèles, et exploiter des modèles d'apprentissage, la CAH et le K-Means, pour en apprendre davantage sur les données.